

RIWI CORP — Simulacro RiwiSupply S.A.S. — HOJA DE RESPUESTAS

SECCIÓN 1 • Conceptos Fundamentales

1. B) Identifica de forma única cada fila de una tabla y no permite valores NULL ni duplicados.
2. C) NOT NULL
3. C) FOREIGN KEY — `supplier_id` referencia la PK de `riwi_suppliers`, por lo tanto es una clave foránea.
4. B) UNIQUE acepta un solo NULL y permite múltiples restricciones UNIQUE por tabla; PRIMARY KEY no acepta ningún NULL y solo puede existir una por tabla.
5. C) Elimina automáticamente todos los registros hijos cuando se elimina el registro padre (propagación en cascada).

SECCIÓN 2 • Normalización

6. C) Cada celda debe contener un único valor atómico y no deben existir grupos repetitivos.
7. C) 3FN — `nombre_proveedor` no depende directamente de `id_registro`, sino del proveedor (dependencia transitiva: `id_registro → id_proveedor → nombre_proveedor`); las variaciones de formato son síntoma de esa redundancia.
8. C) 3FN. Se soluciona creando una tabla `riwi_suppliers` con `id_supplier` como PK y `ciudad` como atributo de esa tabla.

9. Ejercicio de normalización

(a) Problemas de 1FN: Estructuralmente cada celda ya contiene un valor atómico (no hay listas ni grupos repetitivos), pero existen inconsistencias de formato/mayúsculas (`FERRETERIA TORRES SAS`) vs (`Ferretería Torres S.A.S`) que hacen que el mismo proveedor se represente de formas distintas, dificultando identificar la clave real de la entidad.

(b) Dependencias parciales (2FN): No aplican, porque la clave primaria (`id_reg`) es simple (no compuesta); las dependencias parciales solo pueden existir con claves compuestas.

(c) Dependencias transitivas (3FN):

- $(\text{nombre_proveedor}, \text{ciudad_proveedor})$ dependen del proveedor, no directamente de $(\text{id_reg}) \rightarrow (\text{id_reg} \rightarrow \text{id_proveedor} \rightarrow \{\text{nombre_proveedor}, \text{ciudad_proveedor}\})$.
- $(\text{nombre_producto}, \text{categoria}, \text{unidad_medida}, \text{precio_unit})$ dependen del producto, no directamente de $(\text{id_reg}) \rightarrow (\text{id_reg} \rightarrow \text{id_producto} \rightarrow \{\text{nombre_producto}, \text{categoria}, \text{unidad_medida}, \text{precio_unit}\})$.

(d) Esquema relacional final normalizado:

```

riwi_cities(id PK, name)
riwi_suppliers(id PK, name, city_id FK→riwi_cities)
riwi_categories(id PK, name)
riwi_products(id PK, name, unit_of_measure, unit_price, category_id
FK→riwi_categories)
riwi_purchases(id PK, fecha, supplier_id FK→riwi_suppliers, product_id
FK→riwi_products, cantidad)

```

10. B) $(\text{riwi_purchases}(\text{id}, \text{fecha}, \text{supplier_id} \rightarrow \text{riwi_suppliers},$
 $\text{product_id} \rightarrow \text{riwi_products}, \text{unit_price}, \text{quantity}))$

SECCIÓN 3 • Modelo Entidad-Relación

11. B) 1:N

12. C) N:M $\rightarrow$ se implementa con una tabla intermedia $(\text{riwi_purchase_details})$.

13. B) Se necesita una entidad $(\text{riwi_inventory_movements})$ con atributos como tipo de movimiento, cantidad y fecha.

14. B) Una bodega tiene un único responsable asignado y un responsable solo puede estar a cargo de una bodega.

15. Diseño del MER

Entidad	Atributos clave (PK)	Cardinalidad con otras entidades
riwi_cities	id	1 riwi_cities — N riwi_suppliers; 1 riwi_cities — N riwi_warehouses
riwi_suppliers	id, FK city_id	1 riwi_suppliers — N riwi_purchases
riwi_categories	id	1 riwi_categories — N riwi_products
riwi_products	id, FK category_id	1 riwi_products — N riwi_purchase_details; 1 riwi_products — N riwi_inventory_movements
riwi_warehouses	id	1 riwi_warehouses — N riwi_inventory_movements
riwi_purchases	id, FK supplier_id	1 riwi_purchases — N riwi_purchase_details (tabla puente hacia riwi_products, relación N:M real)
riwi_inventory_movements	id, FK product_id, FK warehouse_id	N riwi_inventory_movements — 1 riwi_products; N riwi_inventory_movements — 1 riwi_warehouses

SECCIÓN 4 • DDL

16. B) `ALTER TABLE ... ADD COLUMN`

17. B) La columna `city_id` es una clave foránea que referencia la tabla `riwi_cities`.

18. Script DDL:

```
sql
```

```

CREATE TABLE riwi_categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT
);

CREATE TABLE riwi_products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    unit_price NUMERIC(10,2) NOT NULL CHECK (unit_price > 0),
    unit_of_measure VARCHAR(30) NOT NULL,
    category_id INT NOT NULL REFERENCES riwi_categories(id),
    stock INT DEFAULT 0
);

CREATE TABLE riwi_warehouses (
    id SERIAL PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE,
    city VARCHAR(100) NOT NULL,
    address TEXT,
    manager_name VARCHAR(100) NOT NULL
);

```

19. Sentencias SQL:

```

sql

-- 1.
ALTER TABLE riwi_suppliers ADD COLUMN email VARCHAR(120) UNIQUE;

-- 2.
ALTER TABLE riwi_warehouses ALTER COLUMN address TYPE TEXT;

-- 3.
DROP TABLE IF EXISTS riwi_temp_data;

```

20. A) La tabla con (PRIMARY KEY (purchase_id, product_id)) es la correcta: usa clave compuesta para modelar la N:M y añade los atributos propios de la relación ((quantity), (unit_price)).

SECCIÓN 5 • DML

21. B) PostgreSQL generará un error de violación de restricción única ((unique_violation)).

22. Sentencias INSERT:

```
sql

INSERT INTO riwi_suppliers (name, nit, city_id, email)
VALUES ('TecnoPartes Ltda', '700345678-5', 3, 'tecnopartes@empresa.com');

INSERT INTO riwi_products (name, unit_price, unit_of_measure, category_id, stock)
VALUES ('Tornillo hexagonal 1/2 pulgada', 1200.00, 'UNIDAD', 1, 100);

INSERT INTO riwi_purchases (purchase_date, supplier_id, total_amount)
VALUES (
    CURRENT_DATE,
    (SELECT id FROM riwi_suppliers WHERE nit = '700345678-5'),
    50 * 1200.00
);
```

23. Sentencias SQL:

```
sql

-- 1.
UPDATE riwi_suppliers
SET email = 'nuevo_email@distribuidora.com', name = 'Distribuidora Electrónica'
WHERE id = 2;

-- 2.
UPDATE riwi_products
SET stock = stock - 10
WHERE id = 3 AND stock >= 10;

-- 3.
DELETE FROM riwi_inv_movs
WHERE movement_type = 'EGRESO' AND movement_date < '2023-01-01';

-- 4.
DELETE FROM riwi_suppliers
WHERE id = 99
    AND NOT EXISTS (SELECT 1 FROM riwi_purchases WHERE supplier_id = 99);
```

SECCIÓN 6 • Consultas SQL

24. B) LEFT JOIN devuelve todos los registros de la tabla izquierda y los coincidentes de la

derecha; INNER JOIN solo los que coinciden en ambas.

25.

sql

```
SELECT p.name AS producto, c.name AS categoria, p.unit_of_measure AS unidad_medida
FROM riwi_products p
JOIN riwi_categories c ON p.category_id = c.id
ORDER BY p.stock ASC;
```

26.

sql

```
SELECT w.name AS bodega, m.movement_type AS tipo, COUNT(*) AS total_movimientos
FROM riwi_inv_movs m
JOIN riwi_warehouses w ON m.warehouse_id = w.id
GROUP BY w.name, m.movement_type
HAVING COUNT(*) > 5
ORDER BY total_movimientos DESC;
```

27.

sql

```
SELECT s.name AS proveedor, c.name AS ciudad,
       COUNT(p.id) AS numero_compras,
       COALESCE(SUM(p.total_amount), 0) AS valor_total
FROM riwi_suppliers s
JOIN riwi_cities c ON s.city_id = c.id
LEFT JOIN riwi_purchases p ON p.supplier_id = s.id
GROUP BY s.name, c.name
ORDER BY valor_total DESC;
```

28.

sql

```

SELECT pr.name AS producto, SUM(d.quantity) AS unidades_totales
FROM riwi_purch_detail d
JOIN riwi_products pr ON d.product_id = pr.id
GROUP BY pr.name
ORDER BY unidades_totales DESC
LIMIT 1;

```

29.

sql

```

SELECT w.name AS bodega, w.city AS ciudad,
       SUM(m.quantity * p.unit_price) AS valor_total_inventario
FROM riwi_inv_movs m
JOIN riwi_warehouses w ON m.warehouse_id = w.id
JOIN riwi_products p ON m.product_id = p.id
GROUP BY w.name, w.city
HAVING SUM(m.quantity * p.unit_price) > 500000
ORDER BY valor_total_inventario DESC;

```

30.

sql

```

SELECT s.id, s.name, SUM(p.total_amount) AS total_comprado
FROM riwi_suppliers s
JOIN riwi_purchases p ON p.supplier_id = s.id
GROUP BY s.id, s.name
HAVING SUM(p.total_amount) > (
    SELECT AVG(total_por_proveedor)
    FROM (
        SELECT SUM(total_amount) AS total_por_proveedor
        FROM riwi_purchases
        GROUP BY supplier_id
    ) AS sub
);

```

31.B) (SELECT p.* FROM riwi_products p LEFT JOIN riwi_inv_movs m ON p.id = m.product_id WHERE m.id IS NULL;

32.

sql

```

SELECT c.name AS categoria,
       COUNT(p.id) AS numero_productos,
       AVG(p.unit_price) AS precio_promedio
FROM riwi_products p
JOIN riwi_categories c ON p.category_id = c.id
GROUP BY c.name
HAVING AVG(p.unit_price) > 10000
ORDER BY precio_promedio DESC;

```

33. B) HAVING

SECCIÓN 7 • Vistas y Transacciones

34. B) Una consulta SQL almacenada que se comporta como una tabla virtual.

35.

```

sql

CREATE VIEW vw_riwi_stock_by_product AS
SELECT p.name AS product_name, c.name AS category,
       p.unit_of_measure, p.stock
FROM riwi_products p
JOIN riwi_categories c ON p.category_id = c.id;

```

36.

```

sql

BEGIN;

SAVEPOINT sp_before_supplier;

INSERT INTO riwi_suppliers (name, nit)
VALUES ('Metales del Pacífico Ltda', '950111222-3');

SAVEPOINT sp_before_product;

-- Intento de inserción de producto asociado.
-- Si el producto ya existiera (p. ej. violación de UNIQUE),
-- se revertiría solo hasta este punto sin perder al proveedor insertado:
-- ROLLBACK TO SAVEPOINT sp_before_product;

COMMIT;

```


37. B) ROLLBACK deshace toda la transacción actual; ROLLBACK TO SAVEPOINT deshace solo hasta el punto de guarda definido, manteniendo la transacción activa.

SECCIÓN 8 • Detección de Errores de Modelado

38. B) No se ha definido PRIMARY KEY y contiene atributos de otras entidades (proveedor, producto, categoría), violando la normalización.

39. C) `riwi_warehouses (1) — (1) riwi_inv_movs` es incorrecta: una bodega registra muchos movimientos de inventario a lo largo del tiempo, por lo que la cardinalidad real es 1:N, no 1:1.

40. Errores encontrados:

1. `id INT` no tiene `PRIMARY KEY` — debe ser `id SERIAL PRIMARY KEY`.
2. `FOREING KEY` está mal escrito — debe ser `FOREIGN KEY`.
3. La referencia `riwi_product(id)` usa el nombre de tabla en singular — la tabla correcta es `riwi_products`.
4. En el `INSERT`, los nombres de columnas se escribieron dentro de `VALUES` en lugar de en la lista de columnas antes de `VALUES`.
5. En el `SELECT`, la cláusula `HAVING` repite una condición de columna (`movement_type = 'ENTRADA'`) que ya fue filtrada en el `WHERE`; `HAVING` debe usarse solo para condiciones sobre funciones de agregación.
6. `ORDER warehouse_id` falta la palabra clave `BY`, y además `warehouse_id` no forma parte del `SELECT`/`GROUP BY` de la consulta.

Versión corregida:

```
sql
```

```
CREATE TABLE riwi_inventory_movements (  
    id SERIAL PRIMARY KEY,  
    movement_date DATE NOT NULL,  
    movement_type VARCHAR(10),  
    quantity INT,  
    product_id INT,  
    warehouse_id INT,  
    FOREIGN KEY (product_id) REFERENCES riwi_products(id),  
    FOREIGN KEY (warehouse_id) REFERENCES riwi_warehouses(id)  
);
```

```
INSERT INTO riwi_inventory_movements  
    (movement_date, movement_type, quantity, product_id, warehouse_id)  
VALUES  
    ('2024-01-15', 'ENTRADA', 50, 1, 2);
```

```
SELECT product_id, SUM(quantity) AS total_quantity  
FROM riwi_inventory_movements  
WHERE movement_type = 'ENTRADA'  
GROUP BY product_id  
ORDER BY product_id;
```